

Pre-training

Первый, самый ресурсоемкий этап обучения. В процессе модель накапливает знания о мире. После уже адаптируется к конкретной задаче путём замены входного слоя и дообучения.

Для предобучения используются след. задачи:

1. Masked Language Modeling (MLM) – скрывается часть слов, модель должна предсказать их по контексту.
2. Causal Language Modeling (CLM) – предсказывание след. токена на основе предыдущих
3. Next Sentence Prediction (NSP) – предсказание, идет ли предложение после другого

Unsupervised learning направлен на восстановление вероятностного распределения данных. В результате pre-training получаем вероятн. модель P_θ .

Alignment

Обучение модели, "содержащей знания о мире",
под решение продуктивных задач.

Задача instruction following: модель должна отвечать на запросы
пользователя и выполнять его "поручения".

Supervised fine-tune (SFT)

Хотим, чтобы у "хороших" ответов ans на запрос $prompt$ была высокая вер-ть $p_{\theta}(ans|prompt)$.

$\{(ans_i, prompt_i)\}_{i=1}^L$ - датасет запросов и "эталонных" ответов.

Решаем задачу:

$$\sum_i \log p_{\theta}(ans_i|prompt_i) \rightarrow \max_{\theta}$$

Проблемы:

1. Дорого размечать данные
2. Сдвиг распред.: на обучении предсказываем токены по "эталонным" предгд. На инференсе предсказываем след. токены по генер. предыдущим.
3. Решаем не ту задачу, которую хотим.

Замечание: максимизируем $\log p_{\theta}(ans_i|prompt_i)$, а не $\log p_{\theta}(ans_i, prompt_i)$. Поэтому необходимо замаскировать промпт при подсчете градиента, он должен течь только через ответ.

RLHF

Постановка задачи прямо говорит модели, что можно генерировать вот это и это, а всё остальное нельзя. Но это неверный подход, ответ на вопрос может быть сильно разным: синонимы, перестановки слов. Лучшее подходит формулировка "модель должна генерировать то, что "порадует" пользователя."

Задача принимает след. вид:

$$\mathbb{E}_{\substack{\text{prompt} \\ \text{ans} \sim p_\theta(\text{ans}|\text{prompt})}} Q(\text{ans}, \text{prompt}) + \beta \text{KL}(p_\theta \| p_0) \rightarrow \max_{\theta}$$

где Q - функционал, оценив. качество модели

$$\text{KL}(p \| q) = \mathbb{E}_{i \sim p} \log \frac{p(i)}{q(i)} \text{ - релюпризация}$$

p_0 - чекпоинт модели после SFT-стадии

В данном случае от параметров модели зависит распр., а не функция.

Раньше решали задачу вида $\mathbb{E}_{i \sim \pi} f_\theta(i) \rightarrow \max_{\theta}$ при помощи градиент. спуска:

$$\begin{aligned} \cdot \text{ "Честный" градиент имеет вид } \nabla_{\theta} \mathbb{E}_{i \sim \pi} f_\theta(i) &= \\ = \mathbb{E}_{i \sim \pi} \nabla_{\theta} f_\theta(i) \end{aligned}$$

- На каждом шаге заменяем "частный" градиент на его Монте-Карло оценку по батчу $\xi_1, \dots, \xi_k \sim \pi$

- SGD - несмещен $\Rightarrow$ правильный подбор k обеспечивает сходимость процесса оптимизации к окрестн. оптимума.

Для нашей же задачи можно ввести следующее:

$$\nabla_{\theta} \mathbb{E}_{\xi \sim \pi_{\theta}} f(\xi) = \mathbb{E}_{\xi \sim \pi_{\theta}} f(\xi) \nabla_{\theta} \log \pi_{\theta}(\xi) \quad (2)$$

Научимся считать SGD для данного функционала:

1. Сэмплируем батч $\xi_1, \dots, \xi_k \sim \pi_{\theta}$
2. Вычисляем $\frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \log \pi_{\theta}(\xi_i) f(\xi_i)$

Как строить Q ?

RL pipeline, PPO (proximal policy optimization)

Даны пара ответов, хотим предсказать, какой больше понрав. пользователю:

$$P(ans_1 > ans_2 | \text{prompt}) = \sigma(r_{\theta}(ans_1, \text{prompt}) - r_{\theta}(ans_2, \text{prompt}))$$

$\{\text{prompt}\}_{i=1}^L$ - датасет запросов

$\text{ans}_1, \dots, \text{ans}_k$ - варианты ответов

Разметчик для каждой выберет лучшую

$\{(\text{prompt}, \text{ans}_1, \text{ans}_2, y)\}$ - полученной датасет

$$y=1 \Leftrightarrow \text{ans}_1 > \text{ans}_2$$

Пришли к задаче бинарной классификации y по
признаковой описанию $(\text{prompt}, \text{ans}_1, \text{ans}_2)$.
Вобучили модель r_θ на задачу:

$$\frac{1}{N} \sum_{i=1}^N \log \sigma(r_\theta(\text{ans}_w, \text{prompt}) - r_\theta(\text{ans}_l, \text{prompt})) \rightarrow$$

$$\rightarrow \max_{\theta}$$

Итоговый пайплайн:

1. Генерируем моделью ответы на вопросы
2. Используем людей для разметки сгенерированных ответов
3. (До)обучаем reward модель на размеченных данных
4. Используем PPO-алгоритм для обучения нашей модели

5. Возвращаемся к шагу 1. с обновлением чекпоинтом модели

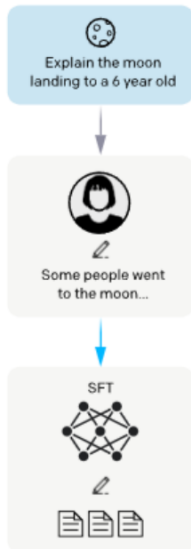
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

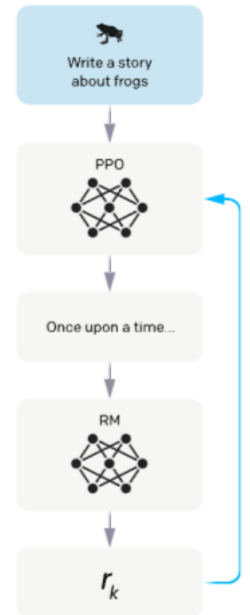
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



(Вместо REINFORCE используется PPO, из-за плохой сходимости первого)

Такой подход даёт существенный прирост в качестве, но имеет след. недостатки:

1. Необходимо хранить сразу 4 модели в памяти (обсудили лишь 2, ещё 2-е в разном)

2. На каждом шаге генерируем ответы, что в авто-регрессионном режиме происходит очень медленно

3. Обучение не стабильно

DPO

у регуляризированной задачи p есть явное решение p_* :

$$p_*(a|p) = \frac{1}{Z(p)} p_0(a|p) \exp\left(\frac{1}{\beta} Q(a, p)\right)$$

где Z - нормир. константа:

$$Z(p) = \sum_a p_0(a|p) \exp\left(\frac{1}{\beta} Q(a, p)\right)$$

Z не может быть вычислено, т.к. суммирование берётся по всем возможным ответам a . Явный вид p_* поэтому не получ.

В текущий момент SOTA явл-ся след. подход:

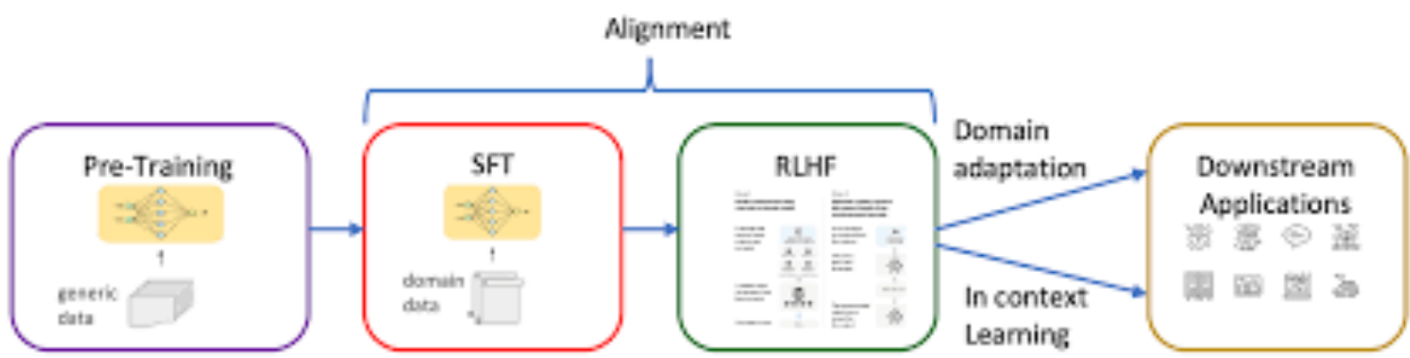
$$\sum_i \log \left(\underbrace{\beta \log \frac{p_0(ans_i | prompt_i)}{p_0(ans_i' | prompt_i)}}_{Q - \beta \log Z(p)} - \beta \log \frac{p_0(ans_i | prompt_i)}{p_0(ans_i' | prompt_i)} \right) \rightarrow \max_{\theta}$$

+ стабильней сж-ся

+ всего один гиперпараметр β

+ нужно хранить 2 штекера вставки

+ не нужно агрегировать ответы, поэтому каждый шаг обучения происходит быстрее



Итоговый pipeline для alignment'a содержит SFT и RLHF стадии. На этапе SFT модель учится генерировать приемлемые ответы: мы, опираясь на эталонные ответы, показываем как правильно отвечать на вопросы/запросы пользователя. На этапе RLHF мы "донастраиваем" распределение ответов так, чтобы оно "удовлетворяло" пользователя.

Были попытки отказаться от SFT стадии: DeepSeek-R1 обучалась только на RLHF. При отказе от SFT мы сильно экономим на разметке данных: вместо дорогого написания ответов разметчикам требуется сравнивать пары ответов. Но позже DeepSeek отметили, что при добавлении даже небольшой SFT-стадии в начало дообучения качество возрастает. Поэтому, на данный момент пайплайн содержит обе стадии.

Методы дообучения

Full fine-tune

Если много ресурсов. Позволяет достигать наилучшего качества

Дообучение части слоёв

Есть гипотеза: "уровень абстракции растёт с номером слоя. Поэтому проиндентифицировать признаки можно только через последние слои, в которых вводятся признаки специфичные для данной задачи.

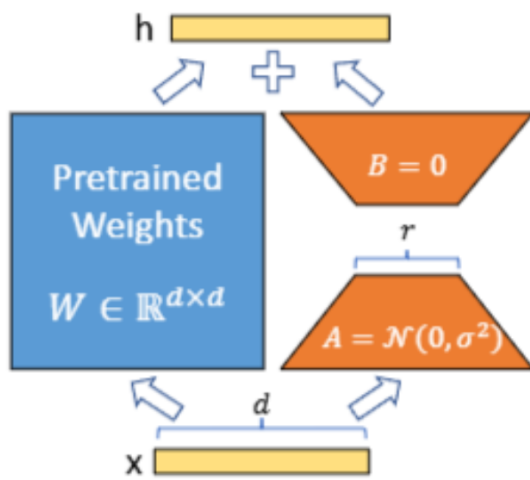
PEFT

Если мало ресурсов

LoRA

Гипотеза, что скрытые представления лежат в окрестности малой размерности. Поэтому добавив к весам $W = W_0 + \Delta W$ можно

искать в классе матриц ранга не более r , где $r \leq d$.
 Для параметризации $\forall W \in \mathbb{R}^{d \times d}$ можно использовать стокетное разложение $\Delta W = AB^T$, где $A, B \in \mathbb{R}^{d \times r}$



Посчитаем сложность умножения матрицы W на вектор x :

1. $z_0 = W_0 x$, сложность d^2
2. $z_1 = B^T x$, сложность dr
3. $z_2 = A z_1$, сложность dr
4. $Wx = z_0 + z_2$, сложность d . Таким образом, добавка ΔW не сильно усложняет прямой проход через LLM.

Вместо вычисления градиента $\nabla_{W, L} \in \mathbb{R}^{d \times d}$ будут вычисляться градиенты $\nabla_A L, \nabla_B L \in \mathbb{R}^{d \times r}$. Это позволяет уменьшить вычислительную сложность обратного прохода, также существенно уменьшаются затраты по памяти.

Одним из основных преимуществ данной стратегии является возможность "встроить" адаптер в исходные веса, честно вычислив $W = W_0 + AB^T$. Поэтому процесс инференса (и его вычислительная сложность) не будет отличаться от процесса инференса исходной модели.

Figure 1: Our reparametrization. We only train A and B .

In-context learning

Обучение на лету без обновления параметров модели.
 Вместо fine-tuning модель получает примеров и ожидаемых ответов в самом промпте.

Zero-shot learning. Модель решает задачу без предоставления примеров, чисто за счёт накопленных знаний.

One-shot learning — должна по одному примеру обобщить правила задачи.

Few-shot learning — получает несколько примеров

Chain-of-Thought (CoT)

Обычные LLM выполняют предсказания на основе вероятностных распределений слов, но иногда сложные задачи требуют разбиения на несколько логических шагов. **Chain-of-Thought (CoT)** — это метод, в котором модель поощряется к генерации пошагового объяснения перед получением ответа. Это особенно полезно в задачах, требующих математических вычислений или сложных рассуждений. Например:

Обычное предсказание:

Вопрос: Если у Анны 3 яблока, а у Боба на 2 яблока больше, сколько всего яблок? Ответ: 5.

Chain-of-Thought:

Вопрос: Если у Анны 3 яблока, а у Боба на 2 яблока больше, сколько всего яблок? Шаг 1: У Боба $3 + 2 = 5$ яблок. Шаг 2: Всего $3 + 5 = 8$ яблок. Ответ: 8.

CoT улучшает интерпретируемость модели и её способность к решению многозадачных логических задач.

